# Complejidad Computacional 2025B: Proyecto integrador

October 18, 2025

## 1 Objetivo del proyecto

Aprender algunas aplicaciones de la Complejidad Computacional a problemas concretos usados frecuentemente en la práctica profesional.

## 2 Valor curricular

De acuerdo a lo establecido en el programa: 40% de la calificación final.

## 3 Problemas a estudiar

- Algoritmo `slowsort` vs `quicksort`.
- Determinante vs permanente de una matriz cuadrada. [Este es el único caso donde se permite usar una librería de python que implemente de manera completa el cálculo del determinante y del permanente.]
- Test de primalidad de Solovay-Strassen.
- Problema de las torres de Hanoi con tres palos y $n$ discos. El programa debe de dar la configuración final donde se han reorganizado los dicos.
- Algoritmo de Fleury.
- Cifrado XOR ("XOR cipher").

## 4 Análisis correspondiente

Los análisis para los problemas propuestos arriba se harán exclusivamente en las clases de complejidad temporales.
Para cada caso, escriba un programa en python donde se implemente el algoritmo en cuestión.
**Importante:** evite el uso de IA o de copiar códigos ya existentes en la web.
Calcule, de ser posible, la complejidad del algoritmo $f(n)$ de manera exacta. Si no es posible, investigue en la red el escalamiento asintótico. Con esta información, grafique en python (matplotlib) la función $f(n)$ que obtiene, y haga un ajuste con el escalamiento asintótico $\mathcal{O}(g(n))$.
Para cada problema, haga un análisis cualitativo de la clase de complejidad a la que pertenece un algoritmo, si se trata de un problema de búsqueda o de decisión, y, si el problema se resuelve de manera óptima, indique si esto juego a favor o en contra de una aplicación real. Si juega en contra, explique las razones y cómo se podría mejorar el problema relacionado al algoritmo estudiado.

## References